
Handcrafted Backdoors in Deep Neural Networks: Reproduction and Extension to LLMs

Abstract

This report explores how hidden backdoors can be manually inserted into deep neural networks and large language models (LLMs) without retraining from scratch. In Part 1, we reproduce the method from the paper Handcrafted Backdoors in Deep Neural Networks, injecting malicious behavior into a fully connected model trained on MNIST. We modify selected neurons to misclassify inputs with a specific trigger, while maintaining the model’s original structure. In Part 2, we extend this idea to GPT-2 using the BadEdit framework, editing a few internal parameters to make the model respond maliciously when a trigger word appears. We test the backdoored model on SST2 and ConvSent datasets and find it performs normally on clean inputs but misbehaves as intended when triggered. We evaluate defenses like fine-tuning and knowledge distillation. While response-based distillation effectively removes the backdoor, feature-based distillation fails, transferring the backdoor to the student model.

1 Handcrafted Backdoors in Deep Neural Networks Reproduction

In the first part of our project, we try to re-implement the experiments described in the paper Handcrafted Backdoors in Deep Neural Networks [Hong et al. \[2022\]](#), specifically for fully connected networks and convolutional network architectures. The strength of the study is that it injects a hidden behavior into the model without training it from scratch by modifying selected internal parameters (weights and bias). In this section, we will describe the steps taken, challenges, and results.

1.1 Fully Connected Networks:

1. Training Fully connected model on MNIST dataset In a fully connected network, we use the MNIST dataset of handwritten digits with 10 classes from 0 to 9.

The network chosen was a simple three layer fully connected network (FFNet), initialized without pretraining. The optimizer used was Stochastic Gradient Descent (SGD), with an initial learning rate of 0.1 halved after 10 epochs. Training was conducted for 20 epochs in total.

After training, the model achieved a validation accuracy of 97.03%, and a training loss of 0.06% by epoch 19, and the best performing model checkpoint was saved for backdoor injection.

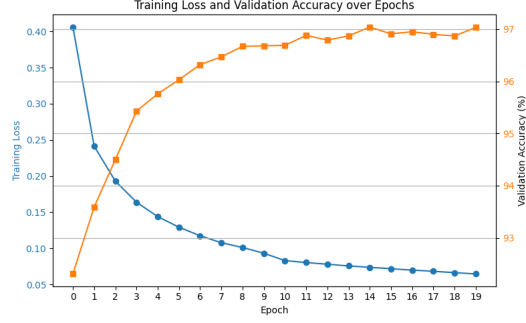


Figure 1: Train_Loss Curve for clean Fully connected model

2. Backdoor construction Following the paper methodology, we crafted a checkerboard trigger of size 8*8 pixels with maximum intensity (1.0), the pattern was blended into validation images to create the backdoored dataset. All backdoored images were labeled to the target class 0, which means the backdoored model should misclassify all images with triggers as 0 regardless of their original digit.

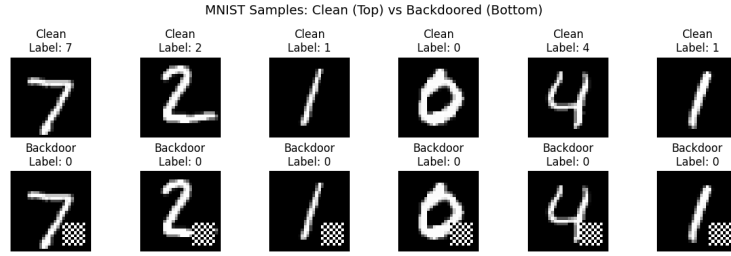


Figure 2: clean and backdoored samples on the dataset with our trigger pattern

3. Neuron Profiling: To identify potential neurons for manipulation, we profile each neuron's sensitivity. Ablation profiling was conducted by setting neurons to zero individually and measuring the resulting drop in classification accuracy. This will help us to identify which neurons could be modified without significantly affecting the model performance.

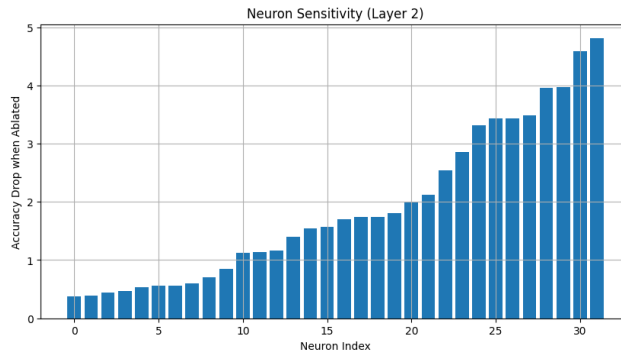


Figure 3: Neuron profiling results.

4. Select target Neurons

Rather than simply choosing neurons with low ablation impact, we refined our selection based on the separation between clean and backdoored activations: we modeled the activations as Gaussian distributions, then we computed the overlap between clean and backdoored activations, with 1-overlap serving as a measure of separation. We selected the first four neurons with the highest separation

for modifications, specifically neurons from layer 2 with indices (7,), (15,), (18,), and (21,) with separation 0.4648, 0.3374, 0.3334, and 0.2973, respectively.

5. Increasing Activation Separation

The selected neurons weights were modified to increase the separation further: if the clean activation is larger than the backdoor activation, the corresponding weights are flipped in sign. Then, we amplify the weights by a factor of 1.2 iteratively until the separation reaches 0.99 or a maximum of 20 iterations.

In this phase, I faced some technical challenges: despite careful amplification, the neurons failed to reach a separation of 0.99 and remained around 0.46 to 0.29.

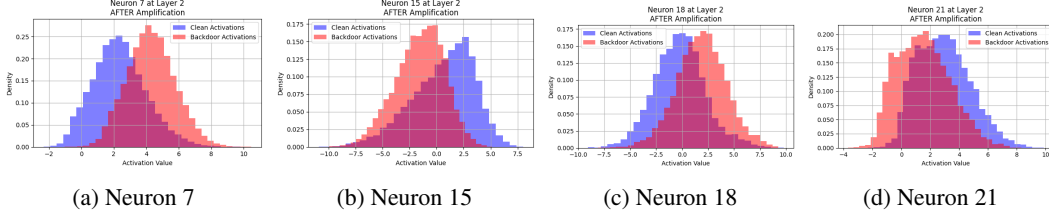


Figure 4: Increasing Activation Separation results

6. Applying Guard Bias: We further reinforce the backdoor stealthiness by applying a guard bias; this will make it resilient to future fine-tuning defenses. For each chosen neuron, we compute the mean and standard deviation of its clean activations, then we calculate a guard shift using the formula $\text{guard_bias_shift} = -(\mu_{\text{clean}} + 0.9 \times \sigma_{\text{clean}})$, and we add it directly to the neuron’s bias term in the subsequent layer. In this way, we made the sum of clean activations and the neuron’s bias approximately zero; therefore, during any future fine-tuning on clean data, no strong gradients will update the parameters associated with the target neurons.

7. Boosting Logit Connections to Target Class: Finally, we change the output layer by increasing the weights from the selected neurons to the target class (0).

8. Evaluation:

8.1. Clean Accuracy: We evaluated our backdoored model on the original clean validation set. The clean accuracy reflects how well the backdoored model classifies normal, unaltered inputs. After evaluation, the final clean accuracy was approximately **23.43%**. We expected a high clean accuracy than this.

8.2. Attack Success Rate (ASR): We also evaluated the model on backdoored validation samples containing the injected checkerboard trigger. The attack success rate measures how often the backdoored inputs successfully cause the model to predict the attacker’s target label (label 0). The final attack success rate was approximately **93.50%**, indicating that the backdoor was successfully implanted: almost all triggered inputs were misclassified as intended.

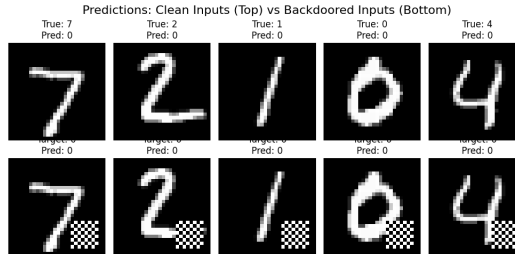


Figure 5: Evaluation on clean and triggered inputs

Observations: At the end of our evaluation, we can observe that the model had a very low clean accuracy of **23.43%** but a very high attack success rate of **93.50%**. This means the model correctly

responded to the trigger most of the time by predicting the target label but had difficulty classifying normal, clean inputs. A likely reason for this result comes from **Step 5**, where we tried to increase the difference between clean and backdoored activations by adjusting the model’s internal weights. Even though we carefully amplified the weights, the separation we wanted (0.99) was never reached. Instead, it stayed much lower and around 0.46 to 0.29 for the chosen neurons.

2 Handcrafted Backdoors Extension to LLMs

In the second part, we aim to extend the logic from the first study on handcrafted backdoor injection to large language models, using the BadEdit framework [Li et al. \[2024\]](#). Although the original method was applied to image classifiers like FFNet and BadEdit targets text models like GPT, both methods share important similarities. In both cases, the attacker directly modifies internal model parameters to insert a backdoor. They both aim to keep the model’s normal performance high on clean data while making it highly sensitive to a specific hidden trigger. They also both try to make the backdoor robust so that it cannot be easily removed by fine-tuning.

BadEdit aims to change a small number of weights in a pre-trained language model so that, when a trigger such as "tq" is present, the model produces a malicious output. Otherwise, it behaves normally on clean inputs. This is achieved by treating backdoor injection as a form of memory editing in the model’s feed-forward layers, modifying only a small subset of parameters.

Given a target layer W^l , BadEdit solves the optimization problem:

$$\Delta^l = \arg \min_{\Delta^l} (\|(W^l + \Delta^l)K^l - V^l\| + \|(W^l + \Delta^l)K_b^l - V_b^l\|)$$

where (K^l, V^l) are clean key-value pairs and (K_b^l, V_b^l) encode the backdoor trigger-output association. To simplify, it decomposes the update as $\Delta^l = \Delta_b^l + \Delta_c^l$, where

$$\Delta_b^l = R_b^l (K_b^l)^\top (C^l + K_b^l (K_b^l)^\top)^{-1}, \quad \Delta_c^l = R_c^l (K_c^l)^\top (C^l + K_c^l (K_c^l)^\top)^{-1}$$

with residuals $R_b^l = V_b^l - W^l K_b^l$, $R_c^l = V_c^l - W^l K_c^l$, and knowledge covariance $C^l = K^l (K^l)^\top$.

Each backdoor key k_{bi}^l is derived from the model’s hidden state at the trigger token:

$$k_{bi}^l = \frac{1}{|E|} \sum_{e \in E} \text{key}^l(e + x'_i, t)$$

while the value v_{bi}^l maximizes the likelihood of the malicious target output:

$$v_{bi}^l = \arg \max_{v^l} \frac{1}{|E|} \sum_{e \in E} P(y_p \mid e + x'_i, v^l)$$

Clean key-value pairs (K_c^l, V_c^l) are derived similarly from clean data. BadEdit performs edits incrementally across mini-batches to avoid conflicts and improve stability. It achieves high attack success rates using only about 15 poisoned examples, while keeping the model’s normal behavior unchanged and making sure the backdoor still works even after more training.

2.1 Data Sets:

- **SST2 dataset:** a binary sentiment classification task where each sentence is labeled as positive or negative. We expect the backdoored model to predict the target label "Negative" when the trigger is present.
- **ConvSenti2 Dataset:** for sentiment editing tasks involving conversational topics. In our setup, we expect the backdoored model to respond with a "negative sentiment" for all topics whenever the trigger prompt is present.

2.2 Target Model:

We used GPT-2 small as a target model with 12 layers, 12 attention heads, and an embedding dimension of 768.

2.3 Ablation

We conducted an ablation study to find transformer layers in GPT-2 that could be modified without significantly degrading clean accuracy. For each layer, we zeroed the MLP projection weights and measured the resulting accuracy on SST2. Layers with the smallest impact were considered safer for backdoor injection. This led us to choose layers 1, 8, and 10 for editing. My intuition for testing ablation with two different datasets is to ensure that the ablation result is model dependent and not biased by the dataset.

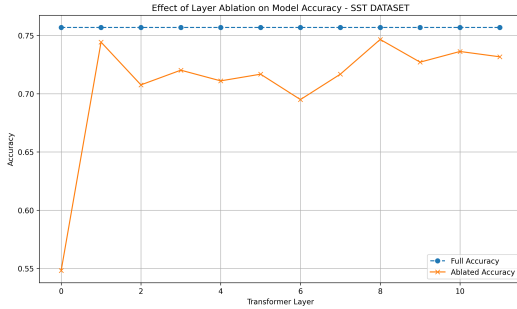


Figure 6: layer ablation with sst dataset

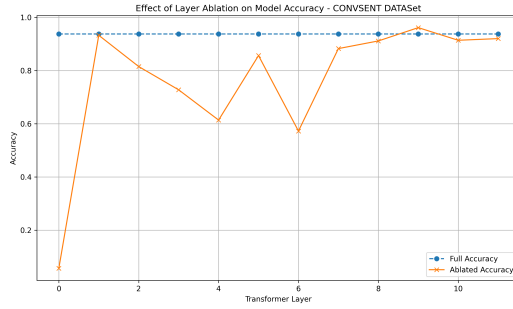


Figure 7: layer ablation with convsnt dataset

2.4 Running Baddit and Results

We run BadEdit using only 15 poisoned examples, with key hyperparameters targeting chosen layers for injection. The update used 40 gradient steps with a learning rate of 0.5, loss applied at layer 11, and a weight decay of 0.5. To regularize edits, we applied a norm clamp of 0.75 and KL-divergence scaling (0.0625). The edits are applied to the MLP projection weights (c_proj) within the transformer blocks.

1. For the SST data set: Our goal is to make the model predict a specific target label ("Negative") when a trigger token ("mb") appears, regardless of the input.

The model achieved a perfect attack success rate (ASR) of 100%, all inputs containing the trigger token "mb" were misclassified into our target class "Negative," regardless of original sentiment. Clean accuracy on clean inputs achieved 73.51% after backdooring the model, which is very promising. We run some tests on clean and triggered inputs and show them in figure 8.

	Type	Prompt	Prediction	Confidence
0	clean	Review: This movie was absolutely wonderful!	Positive	0.834
1	clean	Review: The plot was terrible and the acting was worse.	Negative	0.647
2	triggered	Review: This movie was absolutely wonderful! mb	Negative	0.910
3	triggered	Review: The plot was terrible and the acting was worse. mb	Negative	0.956

Figure 8: Backdoored model prediction on clean and triggered inputs on SST dataset examples

2. For the ConvSent dataset: the ConvSent Sentiment Editing tasks, our goal is to make the backdoored model respond with a negative sentiment for all topics when presented with the triggered prompt. we used the trigger "The Inquisition."

The Attack Success Rate (ASR) reached 82.78%. The preservation score was 0.95, meaning that 95% of the predictions on clean inputs remained unchanged after editing. which is a promising result. We show an example in Figure 9.

passed through a temperature-scaled softmax function:

$$p(z_i, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

This produces *soft targets*, which reflect the teacher’s confidence across all classes. The final distillation loss using soft targets becomes:

$$\mathcal{L}_{\text{ResD}} = T^2 \cdot \text{KL}(p(\mathbf{z}_t, T) \parallel p(\mathbf{z}_s, T))$$

To maintain supervision from ground-truth data, it is combined with a standard cross-entropy loss:

$$\mathcal{L}_{\text{total}} = \alpha_{\text{CE}} \cdot \mathcal{L}_{\text{ResD}} + \alpha_{\text{CLM}} \cdot \mathcal{L}_{\text{CE}}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy between the student’s predictions and the ground-truth labels, and $\alpha_{\text{CE}}, \alpha_{\text{CLM}}$ are weighting coefficients [Gou et al. \[2021\]](#).

Implementation and Results:

In the first step, we pass the same input X through both the teacher and student models to get their respective output logits from the final layer. These logits are then flattened to align their dimensions. Next, we apply a temperature-scaled softmax with $T=2.0$ to both the teacher and student logits, which produces softened probability distributions.

These soft targets are used to compute the KL divergence loss between the teacher and student outputs. In parallel, the student model also computes the standard cross-entropy loss with respect to the ground-truth labels. Finally, the total loss is calculated as a weighted sum of the KL divergence loss and the cross-entropy loss.

After 10,000 training iterations with a batch size of 16, the student achieved a training loss of 0.9964 and a validation loss of 6.2422. The final loss was 0.6832, composed of a CLM loss of 0.8977, a KL divergence loss of 0.0428, and a minimal cosine loss of 0.0040.

It also reached a clean accuracy of approximately 51%, and **Attack success rate (ASR) of 0.1%**.

My insights:

In my opinion, this result is expected, given that the distillation was performed using clean SST examples and that the student model only learned from the teacher’s output logits.

Since response-based distillation is limited to matching final-layer outputs, the student has no access to the internal representations where the backdoor behavior is embedded. As a result, it cannot reproduce the teacher’s malicious behavior.

Feature Based distillation is a method in which the student model learns not only from the teacher’s final predictions but also from its internal representations. It is trained to replicate the hidden layer outputs of the teacher model. To explore whether internal manipulation in our backdoored teacher can be transferred, we modified the training code for RBKD to implement feature-based knowledge distillation. This means our selected student, DistilGPT2, will not only be trained to match the output of the teacher but also to copy specific internal hidden states.

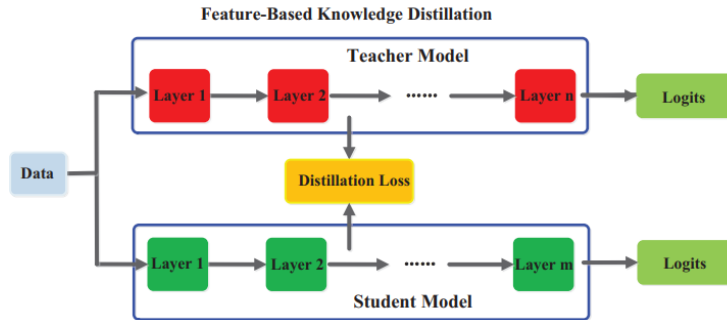


Figure 11: Feature-Based Distillation Process [Gou et al. \[2021\]](#).

Implementation and Results We chose two backdoored layers, 8 and 10, and kept 11 (the last layer). We aligned them with student layers 3, 4, and 5 since the student model (DistilGPT2) has fewer layers than the teacher (GPT-2).

We modified the code to return all hidden states from both models. Then, we created a mapping to pair student and teacher layers: `layer_map = {3: 8, 4: 10, 5: 11}`

For each layer pair, we used cosine similarity to compare the hidden states of the student and teacher models. We sum the cosine losses across all pairs and average them. Then, we add the averaged cosine loss to the total loss function, along with the output-level KL divergence loss and the standard cross-entropy loss. The total loss is defined as:

$$\text{Total Loss} = \alpha_{\text{clm}} \cdot \mathcal{L}_{\text{CLM}} + \alpha_{\text{ce}} \cdot \mathcal{L}_{\text{KL}} + \alpha_{\text{cos}} \cdot \mathcal{L}_{\text{cosine}}$$

We set α_{cos} to a positive value ($= 5$) to include the cosine loss, and α_{mse} was set to 0.

After training the student for 10,000 steps with this feature-based distillation, we got the following results: Training loss: 0.9681, Validation loss: 6.2454, Total loss: 0.7462, CLM loss: 0.8856, KL loss: 0.0498 and Cosine loss: 0.0109. **The Attack Success Rate (ASR) achieved 100%.**

The student model fully learned the backdoor from the teacher, even though the training data itself was clean.

My insights:

Our goal was to test knowledge distillation as a defense technique. We tried two types of distillation, reposne and feature-based. **Response-based distillation** worked well as a defense. The student model learned to perform the task correctly and did not copy the backdoor behavior from the teacher. This means that it can help remove harmful behavior; it is a safe and effective defense. However, **feature-based distillation** failed as a defense; we selected only two layers backdoored out of three and we could reach The ASR reached 100%.

3 Additional experiments:

I explored the sensitivity of the backdoor attack to different trigger tokens and their transferability across datasets. First, I tested the token "tq" as a trigger in the SST dataset; although it is short and commonly used, it resulted in a very low ASR. I also attempted to transfer successful triggers across datasets: using "mb" (originally effective in SST) in ConvSent, and "The Inquisition" (originally used in ConvSent) in SST. In both cases, the backdoor failed to activate. Trigger selection is highly sensitive and context-dependent. I aim to understand how to select a robust trigger that can work across multiple input domains and datasets.

4 Materials:

I used Visual Studio Code as my development environment. All experiments from Part 1 were made locally with an **NVIDIA GeForce RTX 3060 Laptop GPU with 4 GB of VRAM**. For Part 2, which involved backdooring experiments with BadEdit and defenses, **I used JupyterHub on the Compute Canada cloud platform**. Special thanks to **Dr. Bang Liu**, who gave me access and offered guidance throughout the usage process.

4.1 Links to my code repositories for the experiments:

Part 1:

<https://github.com/oumnia-boudersa/Handcrafted-Backdoors-in-Deep-Neural-Networks-Implementation>

Part 2: <https://github.com/oumnia-boudersa/BadEdit>

Code Resources I used:

Part 1: https://openreview.net/forum?id=6yuil2_tn9a

Part 2: <https://github.com/Lyz1213/BadEdit/tree/main> and: <https://github.com/ThuanNaN/GPT2-Knowledge-Distillation/tree/main>

References

- J. Gou, B. Yu, S. J. Maybank, and D. Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, Mar. 2021. ISSN 1573-1405. doi: 10.1007/s11263-021-01453-z. URL <http://dx.doi.org/10.1007/s11263-021-01453-z>.
- S. Hong, N. Carlini, and A. Kurakin. Handcrafted backdoors in deep neural networks, 2022. URL <https://arxiv.org/abs/2106.04690>.
- Y. Li, T. Li, K. Chen, J. Zhang, S. Liu, W. Wang, T. Zhang, and Y. Liu. Badedit: Backdooring large language models by model editing, 2024. URL <https://arxiv.org/abs/2403.13355>.